Microservices Design Patterns — Detailed Notes

Overview

These notes cover migration strategies, data management, and handling complex distributed transactions in microservices architectures.

1 The Strangler Pattern

Context

Used when migrating a legacy monolithic application to microservices.

The Core Concept

Refactoring a massive monolithic application all at once (a "big bang" rewrite) is risky. The Strangler Pattern proposes a gradual migration: build new microservices at the edges of the legacy system, route a fraction of traffic to them, and grow the new services until the monolith can be removed.

How It Works

1. **The Facade (Controller):** Place an API Gateway or controller in front of the legacy application to intercept incoming traffic.
2. **Incremental Migration:** Identify a specific module (e.g., "Search") and rebuild it as an independent microservice.
3. **Traffic Shifting:** Configure the controller to route traffic based on rules. For example:
    - Initially, 90% of traffic goes to the monolith.

- 10% of traffic is routed to the new microservice to test stability.

4. **Rollback Capability:** If the new microservice fails, the controller can switch traffic back to the monolith (0% to the microservice), ensuring no user downtime.
5. **Strangulation:** Gradually increase traffic to the new service (50% $\rightarrow$ 100%). Once stable, delete the corresponding code from the monolith and repeat for other modules.

2 Data Management: Shared DB vs. Database Per Service

Before implementing distributed transaction patterns (e.g., SAGA), decide data topology.

2.1 A. Shared Database (The Anti-Pattern)

Multiple microservices connect to a single monolithic database.

Why it fails:

- **Scaling issues:** High traffic for one service requires scaling the entire database, wasting resources.
- **Coupling:** Schema changes by one service can break others.
- **Technology lock-in:** All services must use the same DB technology even when different stores would be better suited.

2.2 B. Database Per Service (Recommended)

Each microservice owns a private database.

Rules:

- No service may access another service's database directly; communication must use APIs.

Benefits:

- **Loose coupling:** Schema changes are local to a service.
- **Polyglot persistence:** Choose the best database per service (e.g., RDBMS for finance, NoSQL for product catalog).

- **Independent scaling:** Scale each service DB independently.

Challenges:

1. How to maintain transactions across multiple databases? (Solved by **SAGA**.)
2. How to perform joins across multiple databases? (Solved by **CQRS**.)

3 The SAGA Pattern

Goal

Manage distributed transactions and ensure data consistency across multiple services.

Problem: ACID in Distributed Systems

In a monolith, a single DB transaction is ACID. In microservices, a business process often spans multiple services (e.g., Order → Inventory → Payment). If a later service fails after earlier services have committed, you cannot roll back those commits from another database.

Solution: Sequence of Local Transactions

SAGA decomposes a global transaction into a sequence of local transactions.

Success path: Service A updates its DB → emits event → Service B listens and updates → emits event → Service C listens and updates.

Failure path (compensating transactions): If Service C fails, a compensating transaction (an "undo" action) runs in reverse. Example: if a $10 payment fails, the compensating transaction might refund $10 or release reserved inventory — not a DB rollback but a new corrective transaction.

Implementation Approaches

1. Choreography (Decentralized)

- Services communicate via events without a central coordinator.
- **Flow:** Service A emits "OrderCreated." Service B reserves inventory and emits "InventoryReserved." Service C attempts payment on that event.

- **Pros:** Simple to start; no single coordinator bottleneck.
- **Cons:** Hard to track the overall transaction status; risk of cyclic dependencies and complex event chains.

2. Orchestration (Centralized)

- A central orchestrator manages the workflow.
- **Flow:** Orchestrator commands Service A to create order; upon success, instructs Service B to reserve inventory; on failure, commands compensating actions (e.g., undo Order).
- **Pros:** Clear control flow and easier handling of complex workflows.
- **Cons:** Introduces centralized logic that must be maintained and scaled.

4 CQRS (Command Query Responsibility Segregation)

Goal

Solve the "join problem" caused by database-per-service by separating write and read concerns.

Problem

With separate service databases, you cannot perform a single SQL JOIN across services (e.g., Orders in one DB and Customers in another). Performing multiple API calls and joining results in application code is slow and inefficient.

Solution

Split the system into two parts:

Command Side (Write Model): Handles `CREATE`, `UPDATE`, `DELETE`. Uses each service's authoritative database and is optimized for integrity and business logic.

Query Side (Read Model): Handles `SELECT`. Uses a separate read database or materialized view containing denormalized, pre-joined data optimized for fast queries (e.g., a table with Order ID, Product Name, Customer Name).

Synchronization

- When a write occurs, the command side publishes an event (e.g., `UserUpdated`).
- The query side listens to events and updates the read database/materialized view.
- This pattern relies on *eventual consistency*: there may be a small delay (milliseconds to seconds) between a write and the read view update.